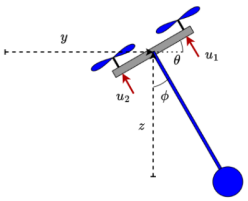
Controls HW 5

Sina Hazeghi

October 8, 2026

1 iLQR

Implement iLQR for swingup and balance of an inverted pendulum on 2D quadrotor system with discretized dynamics $x_{k+1} = f(x_k, u_k)$



Trajectory optimization problem:

$$\begin{aligned} \min_{x,u} \quad & \ell_f(x_N) + \sum_{k=0}^{N-1} \ell(x_k, u_k) \\ \text{s.t.} \quad & x_{k+1} = f(x_k, u_k) \end{aligned}$$

Initial guess is $\hat{u}$; use f to get the $\hat{x}$ trajectory.
To iteratively improve: quadratically approximate the cost-to-go and linearize the dynamics.
Algorithm:

1. Guess $\hat{u} \rightarrow$ get $\hat{x}$ from $f(x, u)$ (forward pass).
2. Compute quadratic cost approximation of $\ell(x, u) \rightarrow$ approximates V.
3. Solve LQR problem to optimize approximation of J (backward pass), then repeat step 1 until the policy stops changing by a lot.

Defn: $\delta x_k = x_k - \hat{x}_k, \quad \delta u_k = u_k - \hat{u}_k, \quad \Delta$ at 1 iteration.
Take original Bellman equation:

$$V(x_k) = \min_{u_k} \ \ell(x_k, u_k) + V\big(f(x_k, u_k)\big)$$

Taylor 2nd-order quadratic approximation of cost becomes:

$$V(x_k) = V(\hat{x}_k + \delta x_k) \approx V(\hat{x}_k) + \delta V_k$$
$$= V(\hat{x}_k) + \underbrace{\nabla V(\hat{x}_k)^T}_{g_k^T} \delta x_k + \tfrac{1}{2}\, \delta x_k^T \underbrace{\nabla^2 V(\hat{x}_k)}_{H_k} \delta x_k \qquad (1)$$

Substituting (1) and linearized dynamics into the Bellman equation:

$$V(x_k) = \underbrace{\ell(\hat{x}_k, \hat{u}_k) + V(\hat{x}_{k+1})}_{V(\hat{x}_k)}$$
$$+ \underbrace{\min_{\delta u_k} \left(\delta \ell_k(\delta x_k, \delta u_k) + \tfrac{1}{2}(A_k \delta x_k + B_k \delta u_k)^T H_{k+1}(A_k \delta x_k + B_k \delta u_k) + g_{k+1}^T (A_k \delta x_k + B_k \delta u_k) \right)}_{\min_u\ \ell(\cdot) + V(\cdot)}$$
$$= \ell(\hat{x}_k, \hat{u}_k) + V(\hat{x}_{k+1}) + \min_{\delta u_k} \delta Q_k(\delta x_k, \delta u_k)$$

Defn δQ_k:

$$\delta Q_k(\delta x_k, \delta u_k) = \frac{1}{2} \begin{pmatrix} \delta x_k \\ \delta u_k \end{pmatrix}^T \begin{pmatrix} Q_{xx} & Q_{xu} \\ Q_{ux} & Q_{uu} \end{pmatrix} \begin{pmatrix} \delta x_k \\ \delta u_k \end{pmatrix} + \begin{pmatrix} Q_x \\ Q_u \end{pmatrix}^T \begin{pmatrix} \delta x_k \\ \delta u_k \end{pmatrix}$$

$$\begin{aligned} Q_x &= \ell_x + A_k^T g_{k+1}, & Q_u &= \ell_u + B_k^T g_{k+1}, \\ Q_{xx} &= \ell_{xx} + A_k^T H_{k+1} A_k, & Q_{uu} &= \ell_{uu} + B_k^T H_{k+1} B_k, \\ Q_{ux} &= \ell_{ux} + B_k^T H_{k+1} A_k, & Q_{xu} &= \ell_{xu} + A_k^T H_{k+1} B_k. \end{aligned}$$

Subscripts u and x are derivatives w.r.t. u and x.

a)

Show $\delta u_k^* = -Q_{uu}^{-1}(Q_u + Q_{ux}\delta x_k) = K_k \delta x_k + d_k$.

$$\delta u_k^* = \arg\min_{\delta u_k}\ \delta Q_k(\delta x_k, \delta u_k)$$

Let $z = \begin{pmatrix} \delta x_k \\ \delta u_k \end{pmatrix}$. Then

$$\nabla_z\, \delta Q_k = \begin{pmatrix} Q_{xx} & Q_{xu} \\ Q_{ux} & Q_{uu} \end{pmatrix} z + \begin{pmatrix} Q_x \\ Q_u \end{pmatrix}$$

$$\nabla_{\begin{pmatrix} \delta x_k \\ \delta u_k \end{pmatrix}} \delta Q_k = \begin{pmatrix} Q_{xx}\delta x_k + Q_{xu}\delta u_k \\ Q_{ux}\delta x_k + Q_{uu}\delta u_k \end{pmatrix} + \begin{pmatrix} Q_x \\ Q_u \end{pmatrix}$$

So

$$\nabla_{\delta u_k} \delta Q_k = Q_{ux}\delta x_k + Q_{uu}\delta u_k + Q_u \overset{\text{set}}{=} 0$$

$$\delta u_k^* = Q_{uu}^{-1}\left(-Q_u - Q_{ux}\delta x_k\right)$$
$$= \underbrace{-Q_{uu}^{-1}Q_{ux}}_{K_k}\delta x_k\ \underbrace{-Q_{uu}^{-1}Q_u}_{d_k} = K_k \delta x_k + d_k$$

b)

Using the form of δu_k^*, find g_k and H_k.

Setup. Original Bellman:

$$V(x_k) = \min_{u_k} \ \ell(x_k, u_k) + V\big(f(x_k, u_k)\big)$$

Applying approximations: $V(x_k) \approx V(\hat{x}_k) + \delta V_k(\delta x_k)$ and $\delta x_{k+1} = A_k \delta x_k + B_k \delta u_k$:

$$V(\hat{x}_k) + \delta V_k = \ell(\hat{x}_k, \hat{u}_k) + V(\hat{x}_{k+1}) + \min_{\delta u_k} \Big(\delta \ell_k + \delta V_{k+1}(A_k \delta x_k + B_k \delta u_k)\Big)$$

Separating out known/constant part from our initial guess:

$$V(\hat{x}_k) = \ell(\hat{x}, \hat{u}) + V(\hat{x}_{k+1})$$

and the part which we are trying to solve for locally approximate trajectory optimization. We see that the part of Bellman we care about solving for each iteration of iLQR is actually:

$$\delta V_k = \min_{\delta u_k} \ \delta \ell_k + \delta V_{k+1}(x_{k+1}) = \delta Q_k(\delta x_k, \delta u_k^*)$$

and recall that we also have approx. for δV_k from 2nd order Taylor expansion:

$$\delta V_k = g_k^T \delta x_k + \tfrac{1}{2}\, \delta x_k^T H_k \delta x_k$$

Expand. Substitute $\delta u_k^* = K_k \delta x_k + d_k$ into δQ_k and group by order in δx_k. The two cross terms $\frac{1}{2}\, \delta x_k^T (Q_{xu} + K_k^T Q_{uu}) d_k$ and $\frac{1}{2}\, d_k^T (Q_{ux} + Q_{uu} K_k) \delta x_k$ are transposes of each other, so they combine:

$$\begin{aligned}
\delta Q_k(\delta x_k, \delta u_k^*) = {} & \tfrac{1}{2}\, \delta x_k^T \big(Q_{xx} + Q_{xu} K_k + K_k^T Q_{ux} + K_k^T Q_{uu} K_k\big) \delta x_k && \text{(quadratic)} \\
& + d_k^T \big(Q_{ux} + Q_{uu} K_k\big) \delta x_k + \big(Q_x^T + Q_u^T K_k\big) \delta x_k && \text{(linear)} \\
& + \tfrac{1}{2}\, d_k^T Q_{uu} d_k + Q_u^T d_k && \text{(constant)}
\end{aligned}$$

Identities.

- $Q_{uu} K_k = -Q_{ux}, \quad Q_{uu} d_k = -Q_u$ (definitions of K_k, d_k)
- $Q_{uu}^T = Q_{uu}, \quad Q_{xu}^T = Q_{ux}$ (Hessian symmetry)
- A scalar equals its transpose

Simplify.

$$\begin{aligned}
\text{Quadratic:} \quad & K_k^T Q_{uu} K_k = -K_k^T Q_{ux} \implies \tfrac{1}{2}\, \delta x_k^T \big(Q_{xx} + Q_{xu} K_k\big) \delta x_k \\
\text{Linear:} \quad & Q_{ux} + Q_{uu} K_k = 0 \implies \big(Q_x^T + Q_u^T K_k\big) \delta x_k \\
\text{Constant:} \quad & \tfrac{1}{2}\, d_k^T \underbrace{Q_{uu} d_k}_{-Q_u} + Q_u^T d_k \ = \ \tfrac{1}{2}\, Q_u^T d_k
\end{aligned}$$

Result. Match terms, using $K_k = -Q_{uu}^{-1}Q_{ux}$ and $d_k = -Q_{uu}^{-1}Q_u$:

$$\begin{aligned} g_k &= Q_x + K_k^T Q_u = Q_x - Q_{xu}Q_{uu}^{-1}Q_u \\ H_k &= Q_{xx} + Q_{xu}K_k = Q_{xx} - Q_{xu}Q_{uu}^{-1}Q_{ux} \\ \Delta V_k &= \tfrac{1}{2}\, Q_u^T d_k = -\tfrac{1}{2}\, Q_u^T Q_{uu}^{-1} Q_u \end{aligned}$$

ΔV_k is the cost improvement from the new controls even when $\delta x_k = 0$. In that case $\delta u_k^* = d_k \neq 0$, so the subsequent states still change and the cost still decreases.

c)

More num. stable to avoid inverting Q_{uu}. Write H_k & g_k in terms of K_k & d_k.

Identities:

$$Q_{uu}K_k = -Q_{ux}, \qquad Q_{uu}d_k = -Q_u$$

From this identity: $Q_{xu} = -K_k^T Q_{uu}$

$$\begin{aligned} H_k &= Q_{xx} - Q_{xu}Q_{uu}^{-1}Q_{ux} \\ &= Q_{xx} + Q_{xu}\underbrace{Q_{uu}^{-1}Q_{uu}}_{I}K_k \\ &= \underline{Q_{xx} - K_k^T Q_{uu}K_k} \end{aligned}$$

$$g_k = Q_x - Q_{xu}Q_{uu}^{-1}Q_u = Q_x + Q_{xu}\underbrace{Q_{uu}^{-1}Q_{uu}}_{I}d_k$$

$$= \underline{Q_x - K_k^T Q_{uu} d_k}$$

d) and e)

For these parts we implement the backward and forward pass of the iLQR method

backward pass: Here we compute K_k, d_k for all the timesteps $k = 0, .., N-2$. To do this we need the values of the coefficients of the Q function δQ_k and to get these values we need g_{k+1} and H_{k+1}

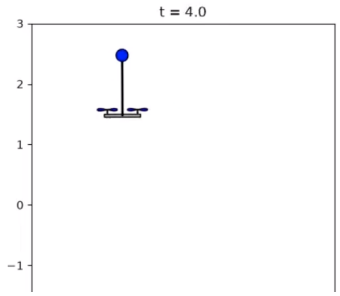
- Luckily at the terminal state we know that $V(x_N) = l_f(x_N)$ and so we can easily compute final g, H by taking gradient and hessian of the terminal loss
- From there we can solve for the Q coefficients $Q_x, Q_u, Q_{ux}, Q_{uu}, Q_{xx}$
- Compute K_{N-2}, d_{N-2} from Q coefficients
- update the g, H with computed K, d, Q

And we simply do this until we go from $k = N-2, ..., 0$

forward pass: We start with $x_0 = \hat{x}_0$ i.e. same as our previous trajectory.

- Using the gains / offsets K_k, d_k we compute δu_k, scaling the offset d_k by α coefficient
  - If α too large then controls will differ too much and our iLQR locally linear / quadratic approximations fall apart
  - If $\alpha = 0$ then new trajectory is exactly old trajectory $\hat{x}_0$, too small it will take a long time to converge
  - I found that $\alpha = 1$ worked fine
- Set the new inputs $u_k = \hat{u}_k + \delta u_k$
- Integrate dynamics forward to new trajectory $x_{k+1} = f(x_k, u_k)$

I ran iLQR and got it to converge and find a solution to quadrotor achieving stable pendulum swingup



2) Direct Collocation

We approximate the state trajectory with cubic splines and the input trajectory with linear splines. Take N sample points $(x_1, u_1), \ldots, (x_N, u_N)$, spaced Δt apart, with $T = (N-1)\Delta t$.

The state spline must pass through the samples:

$$x(0) = x_1, \qquad x(\Delta t) = x_2, \qquad \ldots, \qquad x(T) = x_N.$$

At each knot, the slope of one spline piece must match the slope of the next, and the spline must satisfy the dynamics there:

$$\frac{d}{dt}\ell_1(\Delta t) = \frac{d}{dt}\ell_2(0) = \dot{x}(\Delta t).$$

We also require the dynamics to hold at the midpoint of each segment:

$$\frac{d}{dt}\ell_i\left(t + \tfrac{\Delta t}{2}\right) = \dot{x}\left(t + \tfrac{\Delta t}{2}\right).$$

Together, these make the spline agree with the dynamics at both endpoints and the midpoint of every segment.

(a) Spline coefficients

On the segment from x_i to x_{i+1}, define the cubic

$$s_i(t) = s_0 + s_1 t + s_2 t^2 + s_3 t^3, \qquad t \in [0, \Delta t].$$

Let $f_i = f(x_i, u_i)$. We choose the coefficients so that the spline hits both states and matches the dynamics at both endpoints:

$$s_i(0) = x_i, \qquad s_i(\Delta t) = x_{i+1}, \qquad \dot{s}_i(0) = f_i, \qquad \dot{s}_i(\Delta t) = f_{i+1}.$$

The first condition gives $s_0 = x_i$. The third gives $s_1 = f_i$. The remaining two conditions become

$$x_i + f_i \Delta t + s_2 \Delta t^2 + s_3 \Delta t^3 = x_{i+1}, \tag{2}$$

$$f_i + 2 s_2 \Delta t + 3 s_3 \Delta t^2 = f_{i+1}. \tag{3}$$

Rearranging gives a linear system for s_2 and s_3:

$$\begin{aligned} s_2 \Delta t^2 + s_3 \Delta t^3 &= x_{i+1} - x_i - f_i \Delta t, \\ 2 s_2 \Delta t + 3 s_3 \Delta t^2 &= f_{i+1} - f_i. \end{aligned}$$

Solving (done with SymPy):

$$\begin{aligned} s_2 &= \frac{3(x_{i+1} - x_i) - (2f_i + f_{i+1})\Delta t}{\Delta t^2}, \\ s_3 &= \frac{-2(x_{i+1} - x_i) + (f_i + f_{i+1})\Delta t}{\Delta t^3}. \end{aligned}$$

(b) Value and slope at the midpoint

Evaluating the spline at $t = \Delta t/2$:

$$\begin{aligned} s_i\left(\tfrac{\Delta t}{2}\right) &= x_i + f_i \frac{\Delta t}{2} + s_2 \frac{\Delta t^2}{4} + s_3 \frac{\Delta t^3}{8} \\ &= x_i + \frac{f_i \Delta t}{2} + \frac{3(x_{i+1} - x_i) - (2f_i + f_{i+1})\Delta t}{4} + \frac{-2(x_{i+1} - x_i) + (f_i + f_{i+1})\Delta t}{8} \\ &= \frac{1}{2}(x_i + x_{i+1}) + \frac{\Delta t}{8}(f_i - f_{i+1}). \end{aligned}$$

The slope at the midpoint is

$$\begin{aligned} \dot{s}_i\left(\tfrac{\Delta t}{2}\right) &= f_i + s_2 \Delta t + \tfrac{3}{4} s_3 \Delta t^2 \\ &= f_i + \frac{3(x_{i+1} - x_i) - (2f_i + f_{i+1})\Delta t}{\Delta t} + \frac{3}{4} \cdot \frac{-2(x_{i+1} - x_i) + (f_i + f_{i+1})\Delta t}{\Delta t} \\ &= \frac{1}{\Delta t}\left(-\tfrac{3}{2} x_i + \tfrac{3}{2} x_{i+1}\right) - \tfrac{1}{4}(f_i + f_{i+1}) \\ &= \frac{3}{2\Delta t}(x_{i+1} - x_i) - \frac{1}{4}(f_i + f_{i+1}). \end{aligned}$$

(c) Midpoint constraint

Use the linear input spline on the same segment:

$$r_i(t) = u_i + \frac{t}{\Delta t}(u_{i+1} - u_i),$$

which gives $r_i(\Delta t/2) = \frac{1}{2}(u_i + u_{i+1})$. We want a constraint function

$$h_i\big(s_i(\tfrac{\Delta t}{2}),\, \dot{s}_i(\tfrac{\Delta t}{2}),\, u_i,\, u_{i+1}\big) = 0$$

that forces the spline to obey the dynamics at the midpoint. If the dynamics hold there, then

$$\dot{s}_i\big(\tfrac{\Delta t}{2}\big) = f\big(s_i(\tfrac{\Delta t}{2}),\, r_i(\tfrac{\Delta t}{2})\big)\,,$$

where $s_i(\Delta t/2)$ approximates the true state at the midpoint. Substituting the midpoint input, the constraint is

$$h_i = \dot{s}_i\big(\tfrac{\Delta t}{2}\big) - f\big(s_i(\tfrac{\Delta t}{2}),\, \tfrac{1}{2}(u_i + u_{i+1})\big) = 0.$$

3) Planar Arm Trajectory Optimization

Consider a planar 2-link arm with a ball at the end effector. The configuration and state are

$$q = (\theta_1, \theta_2)^T, \qquad x = (q,\ \dot{q})^T,$$

with $m_e = 1$, link length $L = 1$, and ball mass $m_b = 2$. The arm has actuators at the joints, with limits on torque, joint position, and joint velocity:

$$-u_{\max} \le u(t) \le u_{\max}, \qquad -q_{\max} \le q(t) \le q_{\max}, \qquad -\dot{q}_{\max} \le \dot{q}(t) \le \dot{q}_{\max}.$$

The task is a throwing trajectory over $[0, t_e]$ that minimizes control effort

$$g(u(\cdot)) = \int_0^{t_e} \|u(t)\|^2\, dt$$

subject to the torque and velocity constraints. Using direct collocation, this becomes a nonlinear program:

$$\begin{aligned} \min_{v} \quad & g(v) \\ \text{s.t.} \quad & c(v) = 0, \quad A_{\text{eq}} v = b_{\text{eq}}, \quad \text{lb} \le v \le \text{ub}, \end{aligned}$$

where

$$v = (x_0, \ldots, x_{N-1},\, u_0, \ldots, u_{N-1})^T.$$

At time t_e the ball detaches from the arm and becomes a projectile. We design the trajectory so that the ball lands a distance d away.

(a)

The initial state constrains v through $x_0 = x_{\text{init}}$.

(b)

We need to describe the state of the system at the release time t_f, when the ball leaves the end effector. This means expressing the state x_f at release (where the ball is launched from) in terms of q and $\dot{q}$.

Position of the ball at t_f:

$$p_x = L\sin q_0 + L\sin(q_0 + q_1),$$
$$p_y = -L\cos q_0 - L\cos(q_0 + q_1).$$

Velocity of the ball at t_f (the time derivative of the position):

$$v_x = L\cos q_0\,\dot{q}_0 + L\cos(q_0 + q_1)(\dot{q}_0 + \dot{q}_1),$$
$$v_y = L\sin q_0\,\dot{q}_0 + L\sin(q_0 + q_1)(\dot{q}_0 + \dot{q}_1).$$

Landing condition. After release the ball is a projectile. It lands at the time t_{land} when its height reaches zero:

$$p_y + v_y\,t_{\text{land}} - \tfrac{1}{2}g\,t_{\text{land}}^2 = 0.$$

The horizontal distance traveled then gives the landing distance:

$$d = p_x + v_x\,t_{\text{land}}.$$

(g) Increasing d and solver failures

With the ball released near the top of the reach ($h \approx 2$ m, $v_z \approx 0$), the flight time is $t_{\text{land}} = \sqrt{2h/g} \approx 0.64$ s, so the required release speed is $v_x = d/t_{\text{land}} \approx 7.8$, 15.6, 23.5 m/s for $d = 5, 10, 15$ m. The torque limit (20 N·m) and velocity limit (15 rad/s) cap the energy the arm can supply, so we found solutions only up to $d \approx 5$ m. Beyond that no trajectory satisfies the dynamics, bounds, and landing constraint together, and the solver returns `kInfeasibleConstraints`.

Some things I tried to get around this:

- Made it so final end effector position at t_f (where ball is thrown) is b/w 1.99 and 2
- For final guess set $\dot{q}_0 = 8$ rad/s
- Relaxed the constraint that final ball distance needs to be = d exactly, instead it can land anywhere $\geq d$

So by doing these things I was able to get $d > 10$ working (still might take 10-15 tries with no good guess to get the solver to find a solution)